Sözleşmeler, SOLID Prensipleri & OOP Mimarisi

1.1 `Interface` (Arayüz/Sözleşme)

Sözlük Anlamı: İki bağımsız sistemin birbiriyle nasıl konuşacağını belirleyen, içi boş metot imzalarından oluşan kurallar bütünüdür.

Neden Kullanılır: Bir sınıfa göbekten bağımlı kalmamak (örneğin gerçek şarj istasyonu cihazı olmadan sahte/mock şarj servisiyle testi çalıştırabilmek) için.

```dart
abstract interface class StationContReact{
    Future<void> startCharging(String socketId);
    Stream<double> get wacthLivePower();
}
```

1.2 `implements` vs `extends` vs `with`

1.2.1 `implements` (Arayüz Sözleşmesi)

Sözlük Anlamı: Bir sınıfın, bir şablonun belirlediği kuralları kabul edip **içini sıfırda kendisinin doldurması** zorunluluğudur.

Neden Kullanılır: Farklı sınıflara ortak bir standart/sözleşme dayatmak, çoklu arayüz desteği sağlamak ve kodda bağımlılıkları azaltmak için.

1.2.2 `extends` (Kalıtım/Miras)

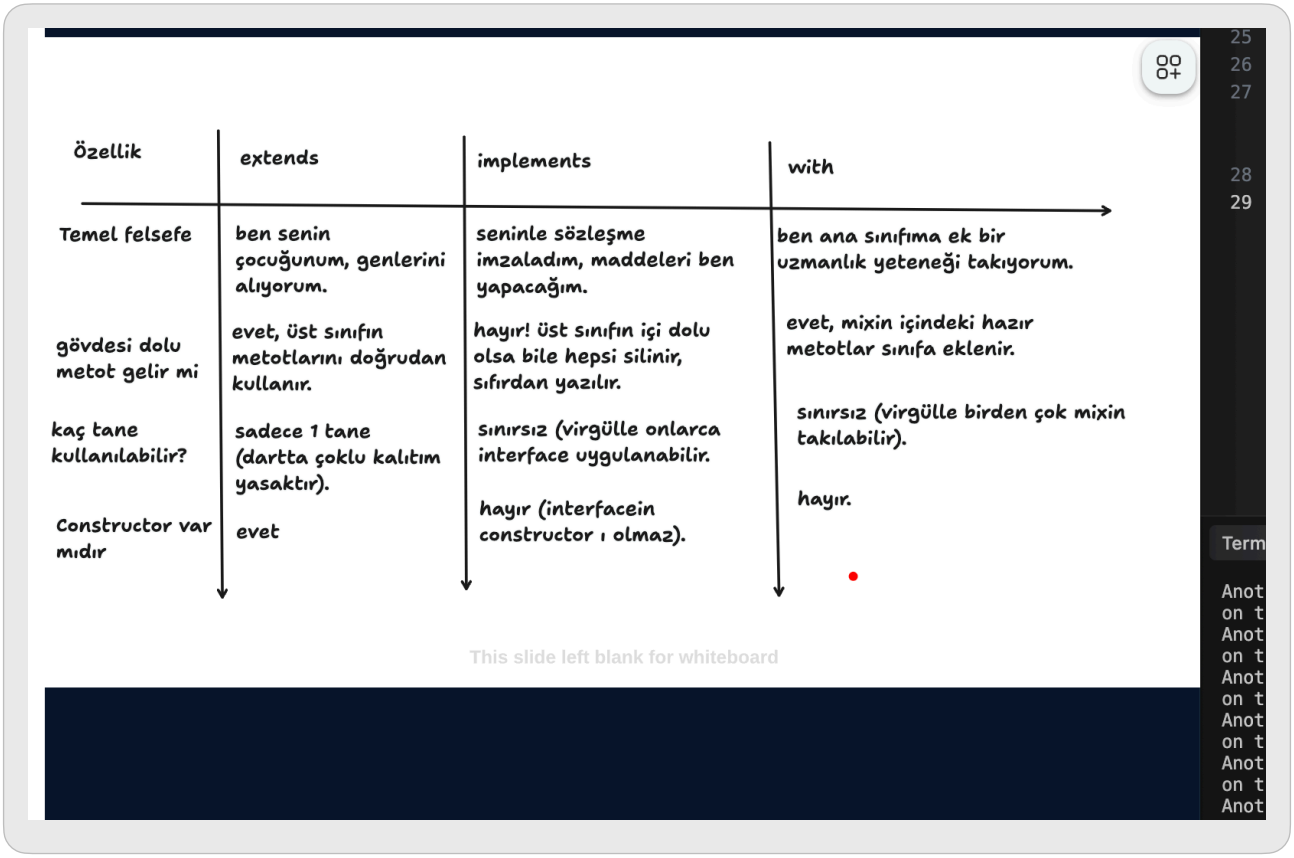
***Sözlük Anlamı:** Bir sınıfım, başka bir sınıfın özelliklerinini ve metotlarını doğrudan devralması.

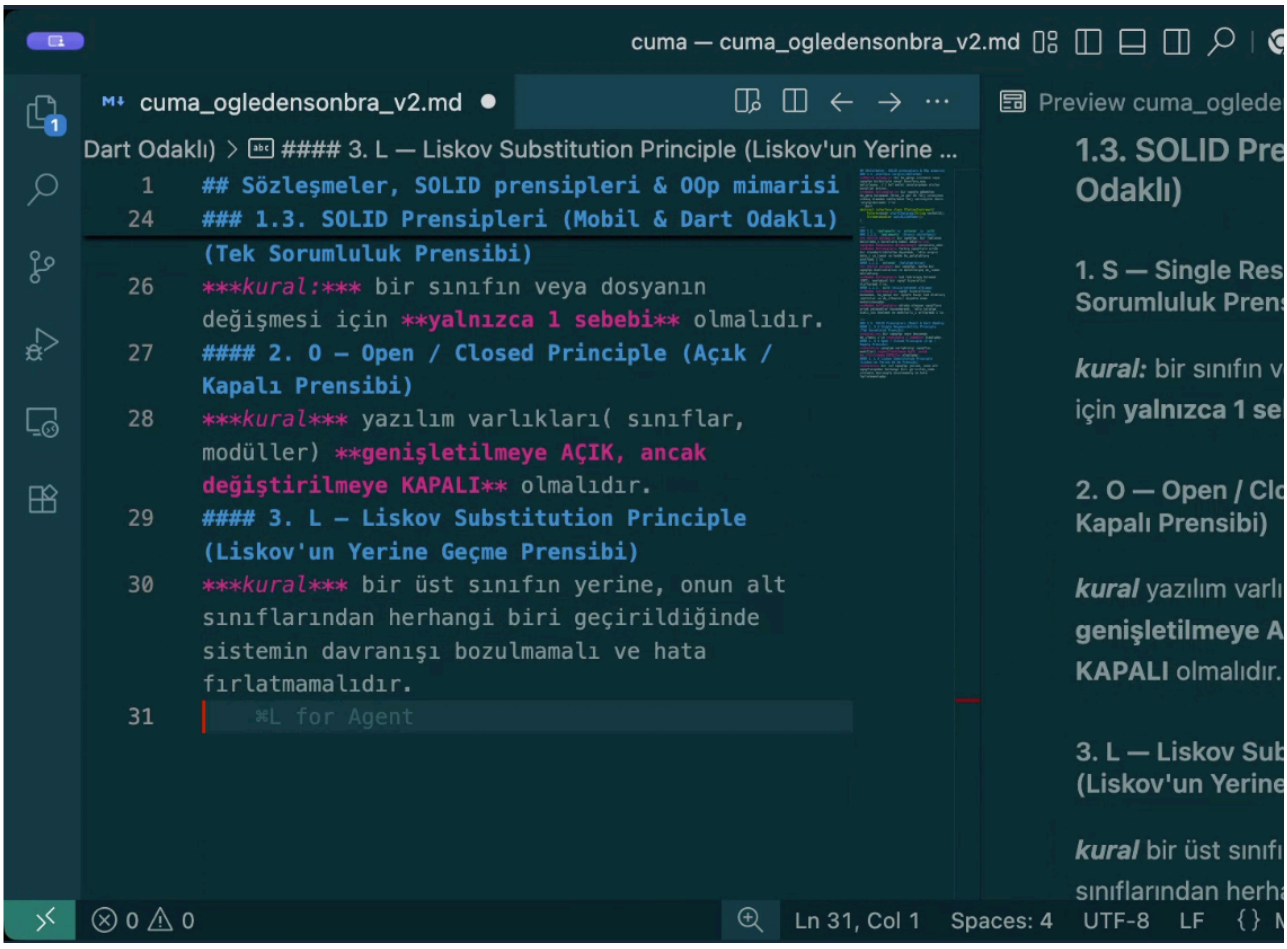
***Neden Kullanılır:** Kod tekrarını önlemek (DRY), mantıksal bir sınıf hiyerarşisi oluşturmak için.

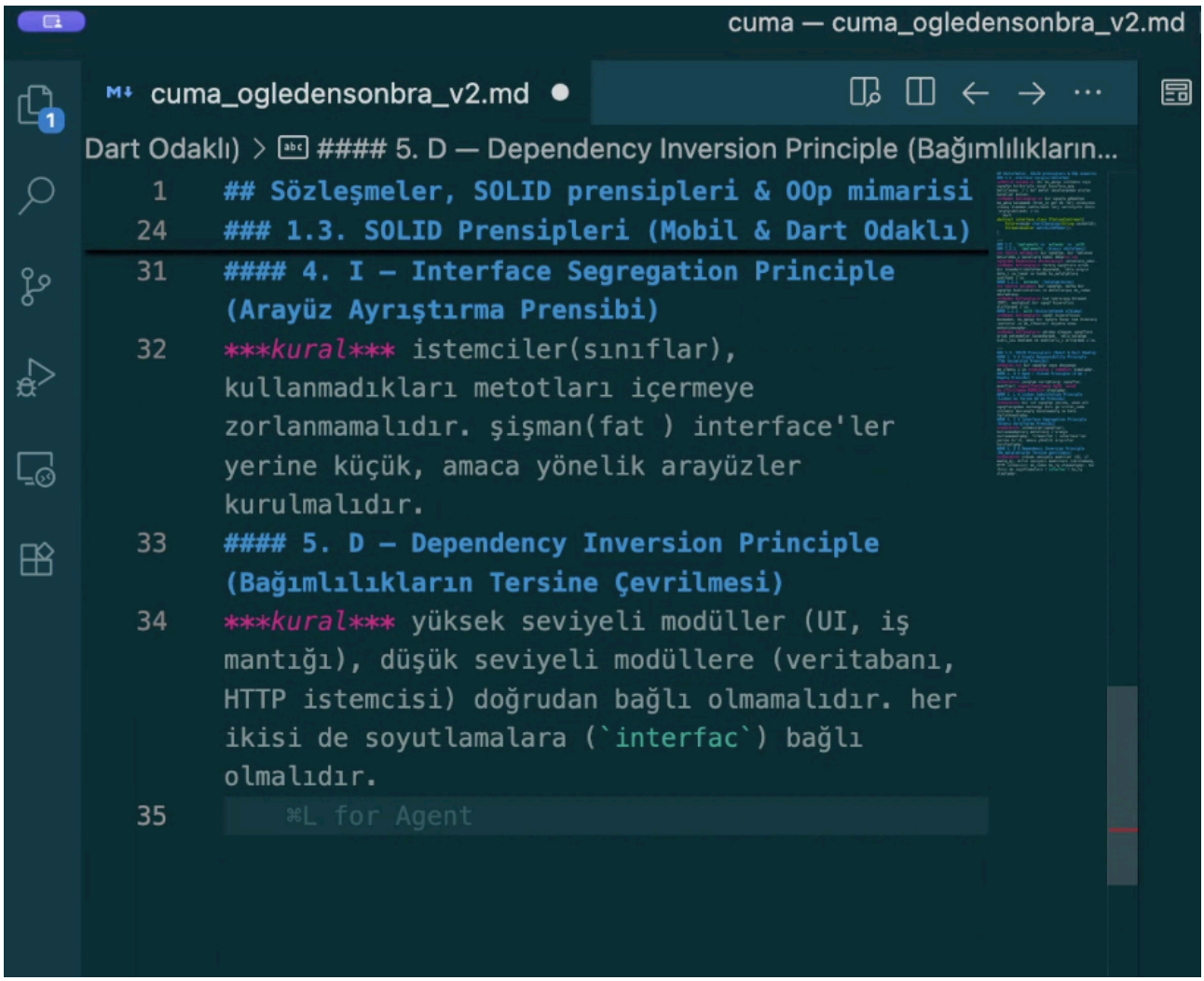
1.2.3 `with` (Mixin/Yetenek Aşılama)

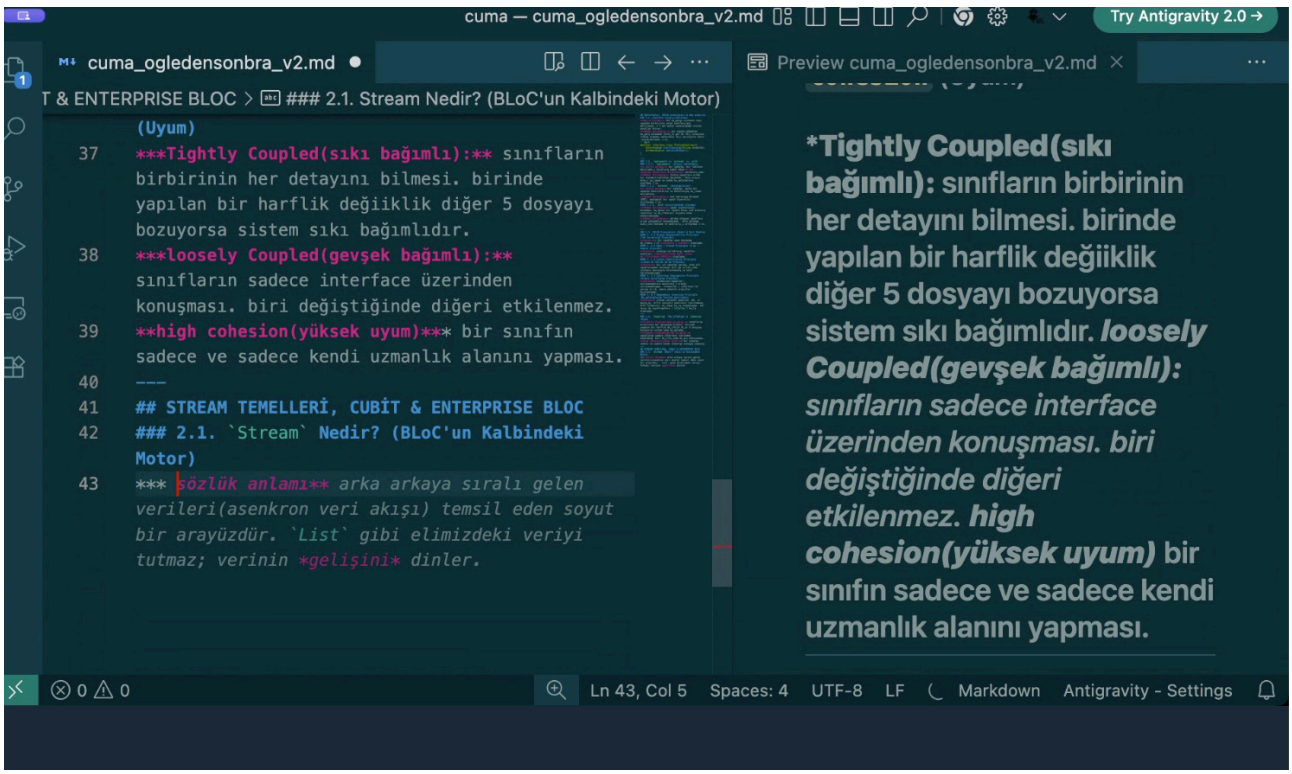
Sözlük Anlamı: Sınıf hiyerarşisni bozmadan, bağımsız bir sınıfa hazır kod blokları (metotlar ve değişkenler) enjekte etme mekanizmasıdır.

Neden Kullanılır: Akraba olmayan sınıflara ortak yetenekler kazandırmak, çoklu kalıtım kimliğini önlemek ve modülerliği artırmak için.

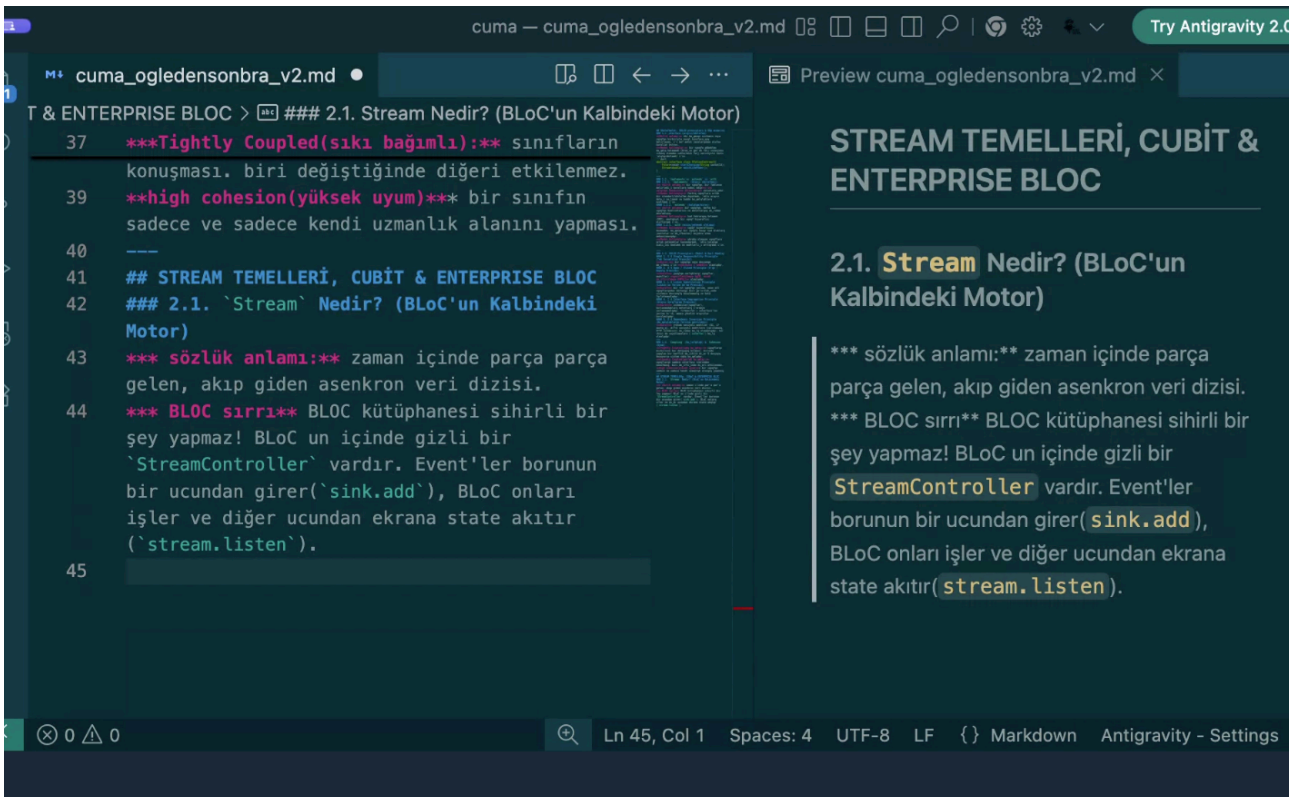

Özellik
extends
implements
with
Temel felsefe
ben senin çocuğunum, genlerini alıyorum.
seninle sözleşme imzaladım, maddeleri ben yapacağım.
ben ana sınıfıma ek bir uzmanlık yeteneği takıyorum.
gövdesi dolu metot gelir mi
evet, üst sınıfın metotlarını doğrudan kullanır.
hayır! üst sınıfın içi dolu olsa bile hepsi silinir, sıfırdan yazılır.
evet, mixin içindeki hazır metotlar sınıfa eklenir.
kaç tane kullanılabilir?
sadece 1 tane (dartta çoklu kalıtım yasaktır).
sınırsız (virgülle onlarca interface uygulanabilir.
sınırsız (virgülle birden çok mixin takılabilir).
Constructor var mıdır
evet
hayır (interfacein constructor ı olmaz).
hayır.
This slide left blank for whiteboard


cuma — cuma_ogledensonbra_v2.md
cuma_ogledensonbra_v2.md
Dart Odaklı) > #### 3. L — Liskov Substitution Principle (Liskov'un Yerine ...
1 ## Sözleşmeler, SOLID prensipleri & OOp mimarisi
24 ### 1.3. SOLID Prensipleri (Mobil & Dart Odaklı)
(Tek Sorumluluk Prensibi)
26 ***kural:*** bir sınıfın veya dosyanın değişmesi için **yalnızca 1 sebebi** olmalıdır.
27 #### 2. O — Open / Closed Principle (Açık / Kapalı Prensibi)
28 ***kural*** yazılım varlıkları(sınıflar, modüller) **genişletilmeye AÇIK, ancak değiştirilmeye KAPALI** olmalıdır.
29 #### 3. L — Liskov Substitution Principle (Liskov'un Yerine Geçme Prensibi)
30 ***kural*** bir üst sınıfın yerine, onun alt sınıflarından herhangi biri geçirildiğinde sistemin davranışı bozulmamalı ve hata fırlatmamalıdır.
31 ⌘L for Agent
Preview cuma_oglede
1.3. SOLID Pre
Odaklı)
1. S — Single Res
Sorumluluk Pren
kural: bir sınıfın v
için yalnızca 1 se
2. O — Open / Clo
Kapalı Prensibi)
kural yazılım varlı
genişletilmeye A
KAPALI olmalıdır.
3. L — Liskov Sub
(Liskov'un Yerine
kural bir üst sınıfı
sınıflarından herha
0 0
Ln 31, Col 1
Spaces: 4
UTF-8
LF

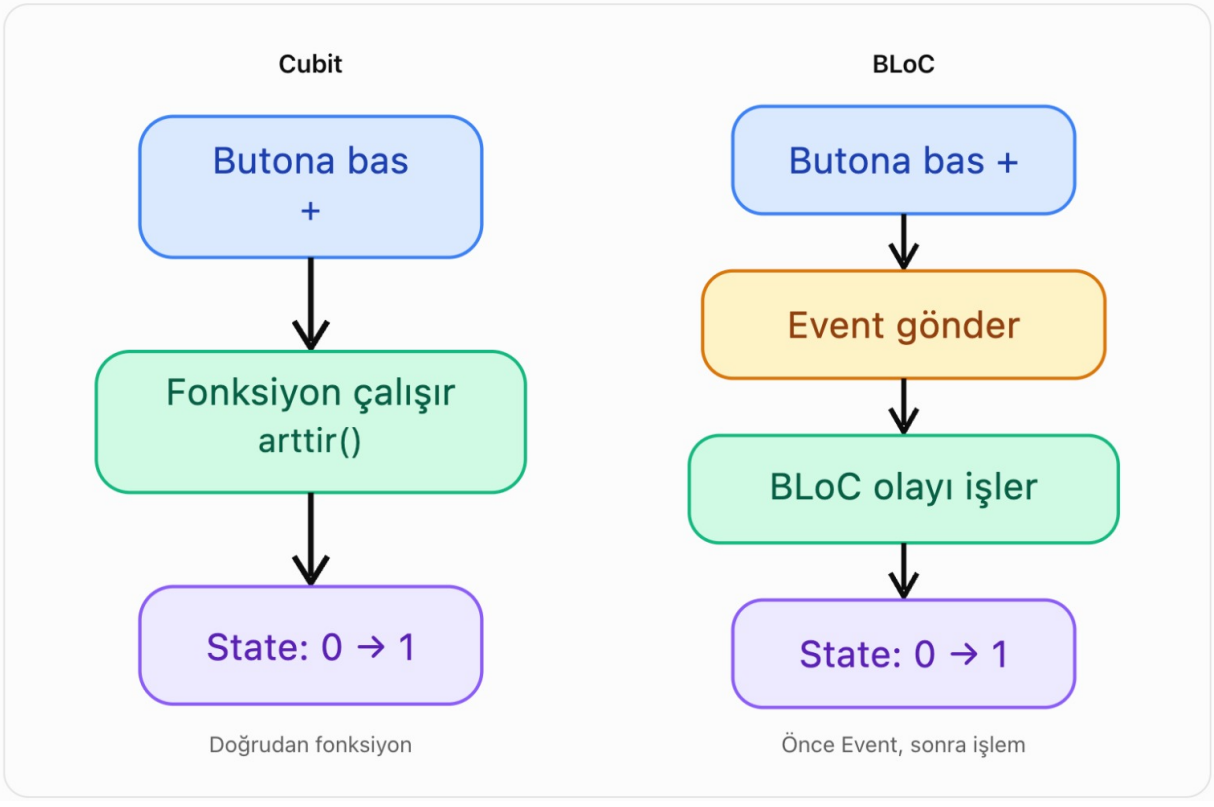




*** block sırrı***block kütüphanesi sihirli bir şey yapmaz blocun içinde gizli bir stream kontroller vardır eventler bu bunun bir ucundan girer sink add bloc onalaır işler ve diğer ucundan ekran a state akıtır stream.listen

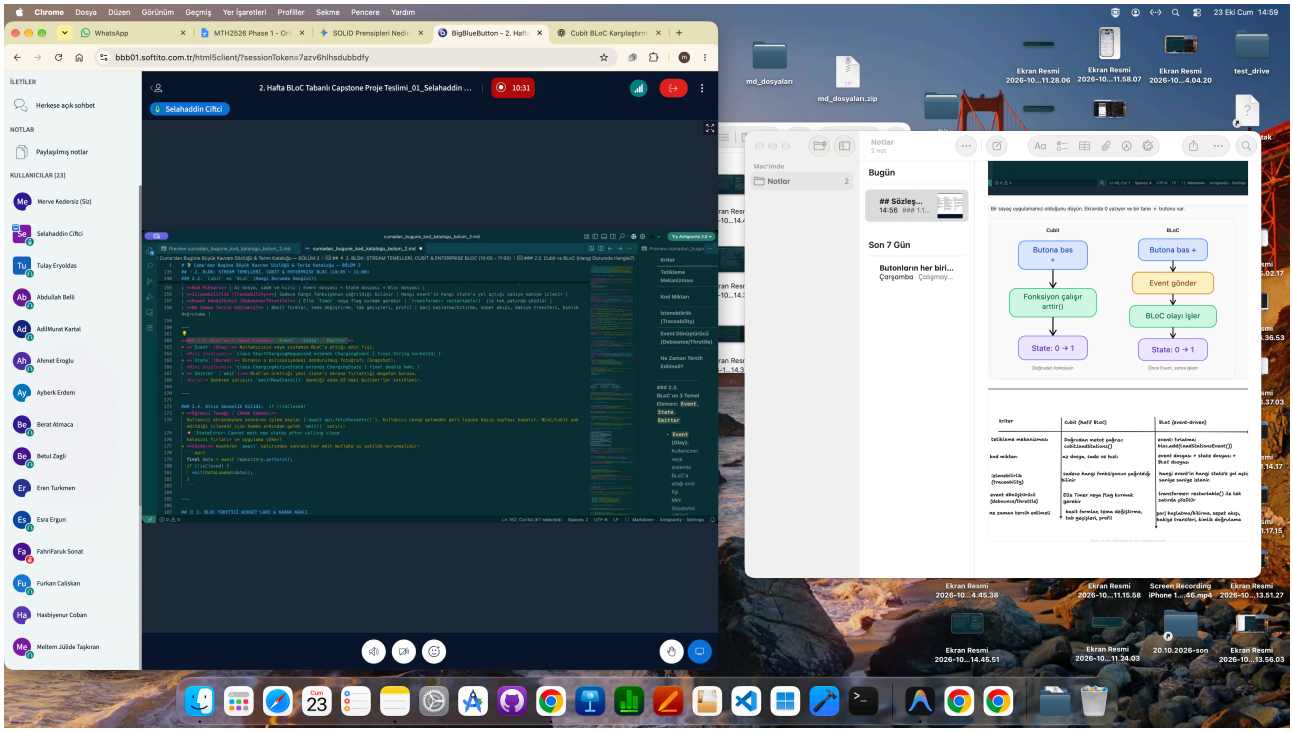


Bir sayaç uygulamamız olduğunu düşün. Ekranda 0 yazıyor ve bir tane `+` butonu var.



kriter	Cubit (hafif BLoC)	BLoC (event-driven)
tetikleme mekanizması	Doğrudan metot çağrısı: cubit.loadStations()	event: fırlatma: bloc.add(LoadStationsEvent())
kod miktarı	az dosya, sade ve hızlı	event dosyası + state dosyası + BLoC dosyası
izlenebilirlik (traceability)	sadece hangi fonksiyonun çağrıldığı bilinir	hangi event'in hangi state'e yol açtı saniye saniye izlenir.
event dönüştürücü (debounce/throttle)	Elle Timer veya flag kurmak gerekir	transformer: restartable() ile tek satırda çözülür
ne zaman tercih edilmeli	basit formlar, tema değiştirme, tab geçişleri, profil	şarj başlatma/bitirme, sepet akışı, bakiye transferi, kimlik doğrulama

This slide left blank for whiteboard



2.3. BLoC'un 3 Temel Elemanı: `Event`, `State`, `Emitter`

*   **Event (Olay):** Kullanıcının veya sistemin BLoC'a attığı emir fişi
    *   *Mini Sözlüğü:* `class StartChargingRequested extends ChargingEvent { final String socketId; }`
*   **State (Durum):** Ekranın o milisaniyedeki dondurulmuş fotoğrafı (Snapshot)
    *   *Mini Sözlüğü:* `class ChargingActive extends ChargingState { final double kwh;`
*   **Emitter ('emit'):** BLoC'un ürettiği yeni state'i ekrana fırlattığı megafon borusu
    *   *Kural:* Senkron çalışır; `emit(NewState())` dendiği anda UI'daki builder'lar tetiklenir

---

2.4. Altın Güvenlik Kilidi: `if (!isClosed)`[cite: 1]

* **Öğrenci Tuzağı / Çökme Sebebi:**
   Kullanıcı ekrandayken asenkron işlem başlar (`await api.fetchSockets()`). Kullanıcı cevap gelmeden geri tuşuna basıp sayfayı kapatır. BLoC/Cubit yok edildiği (`closed`) için hemen ardından gelen `emit()` satırı `StateError: Cannot emit new states after calling close` hatasını fırlatır ve uygulama çöker
* **Çözüm:** Asenkron `await` satırından sonra her `emit` mutlaka şu şekilde korunmalıdır
   ```dart
   if (!isClosed) {
     emit(DataLoaded(data));
   }
   ```

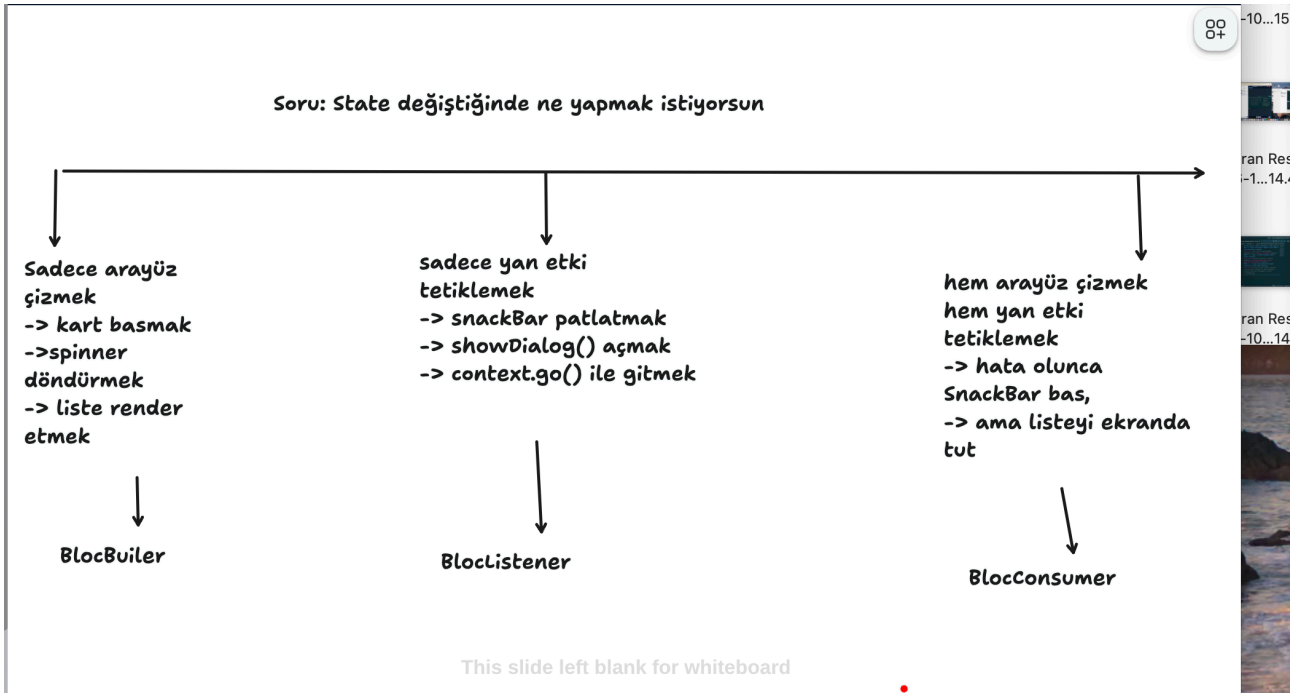
Kullanıcı ekrandayken asenkron işlem başlar (await api.fetchSockets()). Kullanıcı cevap gelmeden geri tuşuna basıp sayfayı kapatır. BLoC/Cubit yok edildiği (closed) için hemen ardından gelen emit() satırı:

💥 StateError: Cannot emit new states after calling close

hatasını fırlatır ve uygulama çöker!

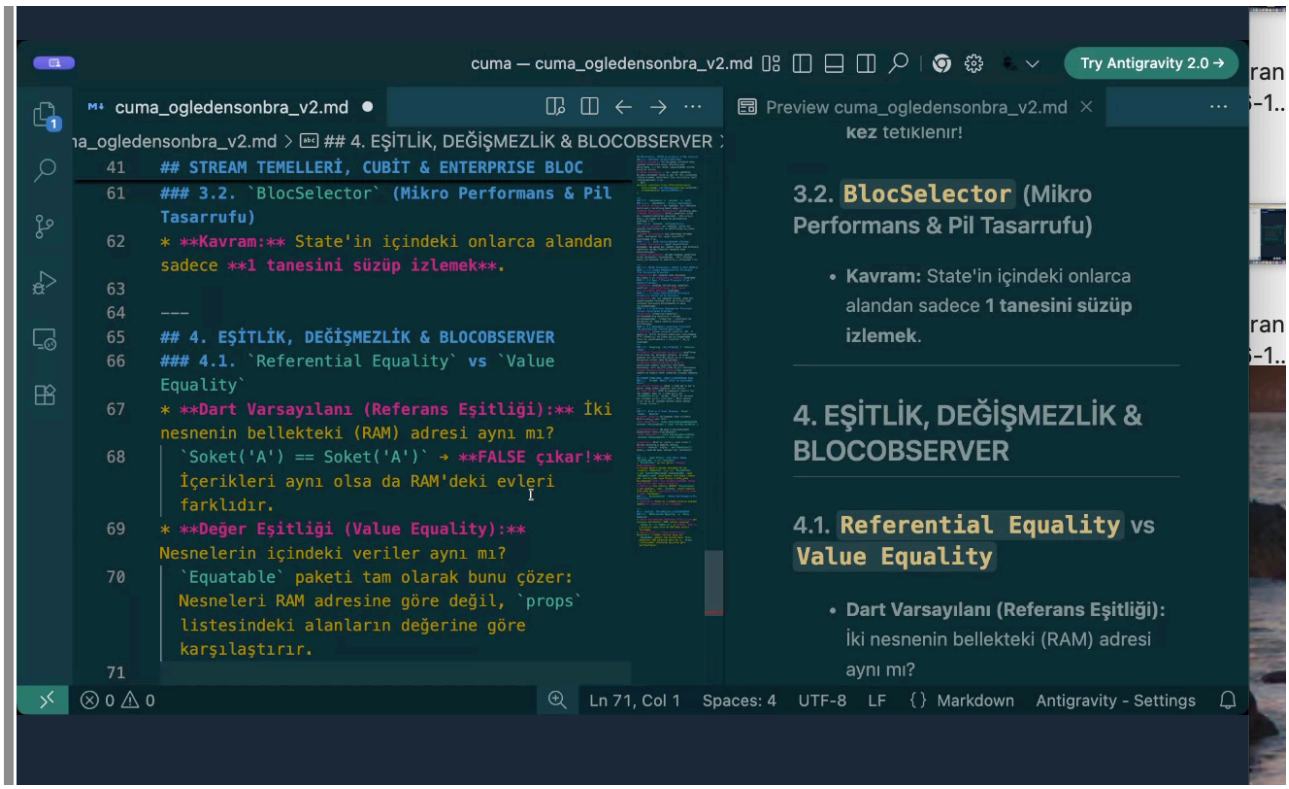
- **Çözüm:** Asenkron await satırından sonraki her emit mutlaka şu şekilde korunmalıdır:final data = await repository.getData();
- if (!isClosed) {
- emit(DataLoaded(data));
- }

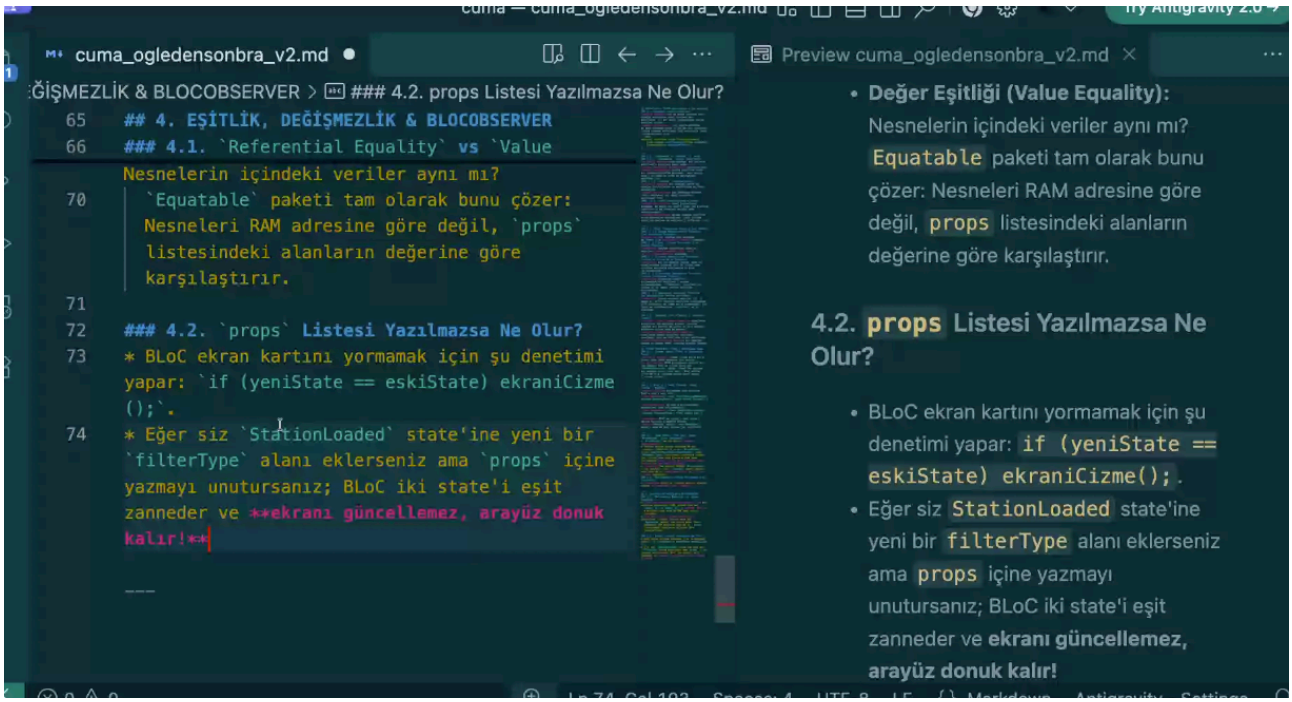
Soru :State değiştinde ne yapmak istiyorsun

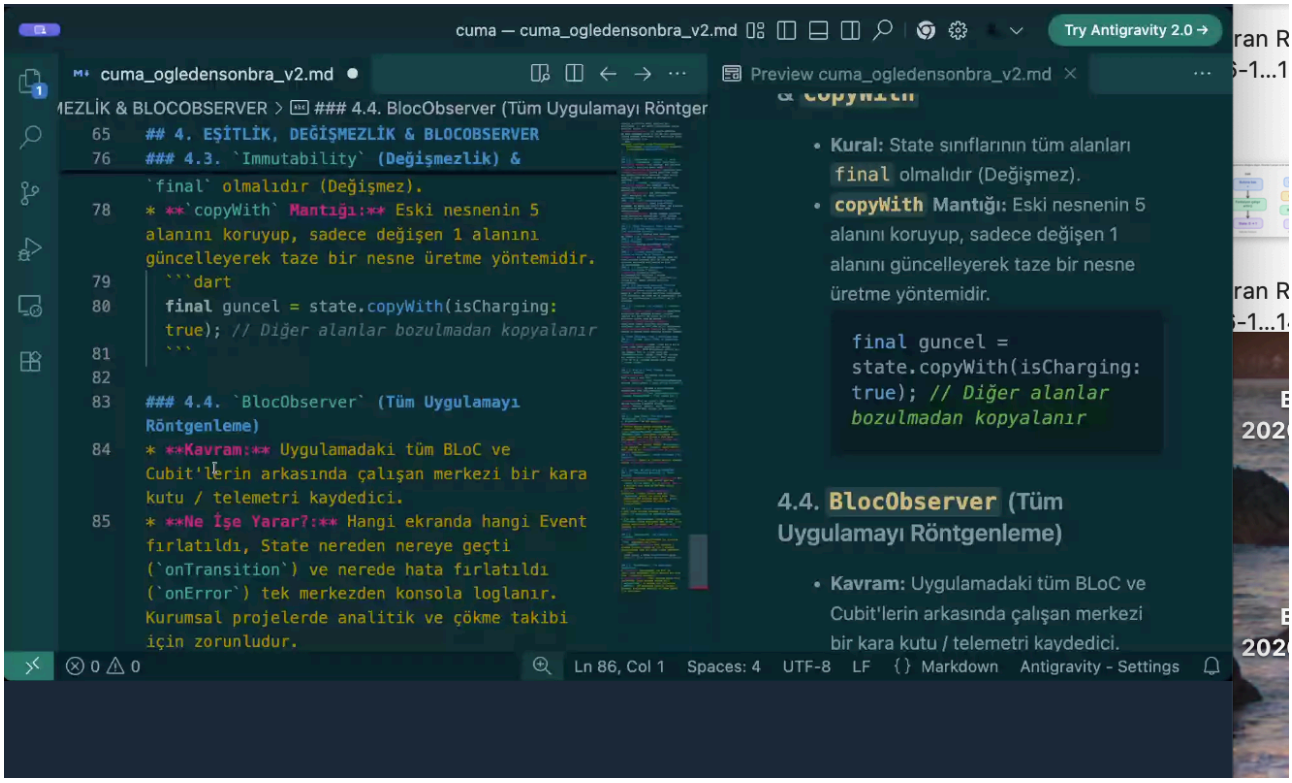


3.1. `Side Effect` (Yan Etki) Neden `BlocBuilder` İçine Yazılamaz?

* `BlocBuilder`'ın tek görevi **Widget döndürmektir**.
* Flutter motoru ekranı saniyede 60 kez çizebilir (Rebuild). Eğer siz `BlocBuilder` içine `ScaffoldMessenger.showSnackBar` veya `Navigator.push` yazarsanız; kullanıcı cihazı yan çevirdiğinde veya klavye açıldığında kullanıcıya **üst üste 10 defa SnackBar patlar veya 10 kez yeni sayfa açılır!**
* **Kural:** Yan etkiler SADECE `BlocListener` içine yazılır; çünkü `listener` sayfa rebuild olduğunda değil, **yalnızca State değiştiğinde 1 kez** tetiklenir!


cuma — cuma_ogledensonbra_v2.md
Try Antigravity 2.0
cuma_ogledensonbra_v2.md
Preview cuma_ogledensonbra_v2.md
4. EŞİTLİK, DEĞİŞMEZLİK & BLOCOBSERVER
41 ## STREAM TEMELLERİ, CUBİT & ENTERPRISE BLOC
61 ### 3.2. `BlocSelector` (Mikro Performans & Pil Tasarrufu)
62 * **Kavram:** State'in içindeki onlarca alandan sadece **1 tanesini süzüp izlemek**.
63
64 ---
65 ## 4. EŞİTLİK, DEĞİŞMEZLİK & BLOCOBSERVER
66 ### 4.1. `Referential Equality` vs `Value Equality`
67 * **Dart Varsayılanı (Referans Eşitliği):** İki nesnenin bellekteki (RAM) adresi aynı mı?
68 `Soket('A') == Soket('A')` → **FALSE çıkar!** İçerikleri aynı olsa da RAM'deki evleri farklıdır.
69 * **Değer Eşitliği (Value Equality):** Nesnelerin içindeki veriler aynı mı?
70 `Equatable` paketi tam olarak bunu çözer: Nesneleri RAM adresine göre değil, `props` listesindeki alanların değerine göre karşılaştırır.
71
kez tetiklenir!
3.2. BlocSelector (Mikro Performans & Pil Tasarrufu)
Kavram: State'in içindeki onlarca alandan sadece 1 tanesini süzüp izlemek.
4. EŞİTLİK, DEĞİŞMEZLİK & BLOCOBSERVER
4.1. Referential Equality vs Value Equality
Dart Varsayılanı (Referans Eşitliği): İki nesnenin bellekteki (RAM) adresi aynı mı?
Ln 71, Col 1
Spaces: 4
UTF-8
LF
Markdown
Antigravity - Settings


cuma_ogledensonbra_v2.md
Preview cuma_ogledensonbra_v2.md
ĞİŞMEZLİK & BLOCOBSERVER > ### 4.2. props Listesi Yazılmazsa Ne Olur?
65 ## 4. EŞİTLİK, DEĞİŞMEZLİK & BLOCOBSERVER
66 ### 4.1. `Referential Equality` vs `Value
Nesnelerin içindeki veriler aynı mı?
70 `Equatable` paketi tam olarak bunu çözer: Nesneleri RAM adresine göre değil, `props` listesindeki alanların değerine göre karşılaştırır.
71
72 ### 4.2. `props` Listesi Yazılmazsa Ne Olur?
73 * BLoC ekran kartını yormamak için şu denetimi yapar: `if (yeniState == eskiState) ekraniCizme();`.
74 * Eğer siz `StationLoaded` state'ine yeni bir `filterType` alanı eklerseniz ama `props` içine yazmayı unutursanız; BLoC iki state'i eşit zanneder ve **ekranı güncellemez, arayüz donuk kalır!**
Değer Eşitliği (Value Equality): Nesnelerin içindeki veriler aynı mı? Equatable paketi tam olarak bunu çözer: Nesneleri RAM adresine göre değil, props listesindeki alanların değerine göre karşılaştırır.
4.2. props Listesi Yazılmazsa Ne Olur?
BLoC ekran kartını yormamak için şu denetimi yapar: if (yeniState == eskiState) ekraniCizme();.
Eğer siz StationLoaded state'ine yeni bir filterType alanı eklerseniz ama props içine yazmayı unutursanız; BLoC iki state'i eşit zanneder ve ekranı güncellemez, arayüz donuk kalır!



1. PRESENTATION (Görsel Katman)
-> UI ekranları, butonlar, theme, BLoC, cubit

↓ bağımlılık içeri bakar

2.DOMAIN (iş kuralları - en kutsal katman)
-> entity (saf veri), UseCase (iş kuralları)
-> Repository Interface(sözleşmeler)
Kural!!! Dış dünyadan (JSON, Dio, SQLite) asla bilgi içermez! sıfır harici paket importu!

↑ sözleşmeyi uygular

3. DATA (veri altyapısı)
->Model /DTO (fromJson/toJson)
->Remote DataSource(dio ile API)
->Local DataSource(Hive / sharedPreferences ile disk)
->Repository Implementation

This slide left blank for whiteboard